

LLM Use Cases: A Practical Appendix

This appendix accompanies the informed-choice briefing. It assumes you’ve decided to experiment and want practical guidance. Each use case includes risks to watch for—not because problems are inevitable, but because awareness of what to monitor produces better outcomes.

A note on discovery

These systems are capable of things you won’t expect—and incapable of things you’d reasonably assume they could do. Even their developers don’t fully understand where the boundaries lie. The only way to discover what works for your purposes is experimentation, and what you discover will change as the technology develops. This is simultaneously one of the frustrations and one of the genuine interests of working with AI: the landscape keeps shifting, and yesterday’s limitations may not apply tomorrow (while yesterday’s reliable capabilities may degrade or disappear). Approach with curiosity rather than fixed expectations.

Contents

- [Getting Started](#)
 - [Administrative and Teaching Tasks](#)
 - [Writing and Revision](#)
 - [Research Support](#)
 - [Multi-Modal Tasks](#)
 - [Data Analysis and Visualization](#)
 - [Advanced Use Cases](#)
 - [Prompting Principles](#)
 - [Verification Strategies](#)
 - [When Not to Use These Tools](#)
 - [Glossary: AI Jargon and Acronyms](#)
-

Getting Started

Choosing a platform

The main consumer platforms are Claude (Anthropic), ChatGPT (OpenAI), and Gemini (Google). All offer free tiers with older models and usage limits; paid subscriptions (€16–25/month) provide access to more capable models and higher limits.

This document defaults to Claude, partly because it’s what I know best and partly because Anthropic’s approach to safety and transparency aligns reasonably with academic values—but that’s a judgment call, not an endorsement. If you’re going to experiment seriously, a paid tier is worth it: the difference between free and frontier models is substantial, and struggling with an underpowered model produces misleading impressions of what the technology can and cannot do.

Current frontier models (late 2025): Claude Opus 4.5, GPT-5.1, Gemini 2.5 Pro. These change; what matters is accessing whatever your chosen platform considers its most capable offering.

Privacy and training settings

By default, many platforms use your conversations to train future models. Most offer settings to opt out.

In Claude: Settings → Privacy → you can disable the use of your conversations for training. With this disabled, your conversations are still processed on Anthropic’s servers (unavoidable), but aren’t incorporated into training data.

This matters for several reasons. You may have principled objections to contributing unpaid labor to commercial AI development. You may work with materials—unpublished research, student work, peer review content—that shouldn’t become training data. Even for routine use, opting out costs you nothing and reduces exposure.

What opting out doesn’t do: it doesn’t make your conversations private in an absolute sense. Data still passes through commercial servers. Don’t input anything you’d be uncomfortable with a company storing, even temporarily. Confidential personnel matters, sensitive student information, embargoed research—these shouldn’t go into any commercial LLM regardless of privacy settings.

Memory settings

Current systems can maintain memory across conversations—tracking your preferences, ongoing projects, and context over time. You can view, edit, and delete what the system remembers.

Whether to enable this depends on your use patterns. If you’re using the tool for ongoing projects where context continuity helps, memory reduces repetitive re-prompting. If you prefer each conversation to start fresh, or if you’re uncomfortable with persistent profiles being built, disable it.

Review memory contents periodically if enabled. Systems sometimes remember things inaccurately or retain information you’d prefer they didn’t.

System prompts and custom instructions

Most platforms allow you to set persistent instructions that shape all conversations. In Claude, this is found in Settings > Profile. These instructions tell the system about your context, preferences, and expectations.

An example for humanities work:

I’m a classicist working on Latin literature. When I ask about primary sources, cite specific passages. Flag when you’re uncertain rather than generating plausible-sounding guesses. Challenge my interpretations rather than agreeing reflexively. Avoid theoretical frameworks I haven’t specifically requested. When I ask for counterarguments, give me the strongest versions, not strawmen.

Tailor this to your discipline and working style. Good system prompts reduce the need to repeat context and calibrate tone in every conversation.

Extended reasoning and other features

Some platforms offer extended reasoning modes for complex problems—the system works through the problem step-by-step before responding. In Claude, this is called “extended thinking.” Use it for multi-step analytical problems, not routine queries.

Tool use (web search, code execution, file creation) is available on most paid tiers. Web search helps with current information but introduces its own reliability questions—the system may find and summarize unreliable sources. Code execution is useful for data analysis and visualization if you can verify the outputs.

Administrative and Teaching Tasks

Drafting routine communications

Calls for papers, meeting announcements, module descriptions, reference letter frameworks. Prompt with specifics: purpose, audience, length, tone, key details to include. Review and edit the output—these are starting points, not finished products.

Risks: Generic, anodyne prose. Factual errors if you include details the system misremembers or invents. Tone mismatches. The system defaults to a kind of corporate enthusiasm that may not suit academic contexts. Always rewrite substantially rather than sending lightly-edited output.

Discussion questions for teaching

“Generate 8 discussion questions for undergraduates on [this passage/topic], ranging from comprehension to interpretation.” Select and revise the useful ones; discard the rest.

Risks: Questions that miss disciplinary specificity—generically “literary” questions for a classics text that ignore what classicists actually care about. Surface-level interpretive questions that don’t push toward genuine difficulty. The system doesn’t know your students or your pedagogical goals.

Reformatting and data cleaning

Bibliography format conversion, cleaning OCR output into structured data, transforming messy spreadsheets. These are tasks where verification is trivial but manual work is tedious.

Risks: Subtle errors in bibliographic details (wrong page numbers, slightly mangled titles). For OCR cleaning, the system may “correct” unusual but accurate readings into common but wrong ones. Check outputs against originals, especially for names, dates, and technical terms.

Writing and Revision

Draft feedback

“This paragraph feels unclear—suggest three ways to restructure it.” “This section is too jargon-heavy—rewrite for an educated non-specialist while preserving analytical precision.” “What’s the weakest link in this argument?”

Risks: The system may smooth away productive difficulty or disciplinary precision in favor of generic clarity. Its suggestions optimize for readability in ways that may flatten your argument. Treat suggestions as prompts for your own revision, not as improvements to accept.

Argument testing

Present your argument and ask: “What are the strongest counterarguments? What alternative explanations must I address? What would a scholar from [different methodological tradition] emphasize?”

Risks: The system generates plausible-critique-shaped text, not actual engagement with your argument’s logic. Counterarguments may be superficial, miss the real vulnerabilities, or attack positions you don’t actually hold. Use outputs as checklists for your own thinking—have I considered this angle?—not as authoritative critique. The absence of a counterargument from the output doesn’t mean your argument lacks that vulnerability.

Grant application review with personas

Before submitting a grant application, you can use personas to simulate different reviewer perspectives. Ask the system to adopt specific roles and critique your draft from that standpoint:

“Read this grant application as a panel member who is expert in [your methodology] and skeptical of [your theoretical framework]. What weaknesses would you identify? What questions would you raise in panel discussion?”

“Now read as a reviewer who is sympathetic to the project’s aims but concerned about feasibility and value for money. Where would you push back?”

“Read as someone from an adjacent discipline who doesn’t share the assumptions of [your field]. What would be unclear or unconvincing?”

“Read as a reviewer focused on impact and knowledge exchange. Where is the case weakest?”

Run through several personas, each generating critique from a different angle. Compile the concerns raised as a checklist for revision.

Risks: The critiques may be generic rather than specific to your proposal’s actual weaknesses. The system doesn’t know what real reviewers in your field actually care about—it generates reviewer-shaped criticism based on patterns, which may miss discipline-specific concerns or fixate on issues that wouldn’t actually trouble a panel. Persona-based critique can create false confidence that you’ve stress-tested the application when you’ve only received plausible-sounding feedback. The personas may also be too gentle; real reviewers operating under time pressure with many applications to assess can be more brutal and more idiosyncratic than the system’s simulations.

Use this as one input among several. It doesn’t replace feedback from colleagues who know the funding landscape, have sat on panels, or have expertise you lack. But it can surface angles you hadn’t considered and help you anticipate obvious objections before others see the draft.

Exploring alternative framings

“Present three different interpretations of this evidence, from different theoretical frameworks.” “How might this argument be framed for [different audience]?”

Risks: Theoretical frameworks may be presented superficially or conflated. The system reproduces general patterns, not the specific texture of how a particular tradition actually approaches problems. Useful for brainstorming, not for understanding those traditions.

Research Support

Literature review scaffolding

“Based on these sources I’m consulting, what themes and debates seem central? What potential gaps might I be missing?” Use this to structure your own reading, not to substitute for it.

Risks: The system may identify themes that sound plausible but don’t reflect how the field actually organizes itself. It may miss what matters and emphasize what’s superficially prominent. Verify thematic analysis against your actual engagement with sources. The system hasn’t read these works—it’s pattern-matching on titles, abstracts, and whatever fragments appear in training data.

Problem-solving strategies

“I need to approach [complex research problem]. What methodological strategies should I consider, and what are the evidentiary requirements and limitations of each?”

Risks: Strategies may be generic rather than fitted to your specific problem. The system may miss approaches standard in your subfield or suggest approaches that wouldn’t occur to a specialist because they’re not actually viable. Use as a starting point for structuring your own research plan, then consult colleagues or methodological literature.

Code for digital projects

“Write Python code to extract place names from these TEI-encoded texts and count frequency.” “Create a visualization of this dataset.” Systems with code execution can run and debug code directly.

Risks: Code may work on test cases but fail on edge cases in your actual data. Logic errors may produce plausible-looking but wrong results. If you can’t read the code well enough to verify its logic, you can’t verify its outputs. For anything substantive, have someone code-literate review it, or develop enough facility to check the logic yourself.

Multi-Modal Tasks

OCR and transcription

Upload images of manuscripts, inscriptions, or archival documents. “Transcribe this text and flag uncertain readings.”

Risks: The system will produce confident transcriptions that contain errors, especially with damaged materials, non-standard scripts, or abbreviations. It may normalize unusual spellings or silently resolve ambiguities. Treat outputs as first passes requiring careful verification against originals. For anything you’ll cite or publish, manual verification is essential.

Image-based queries

Upload photographs of artifacts, sites, architectural features. “What can you tell me about this object’s typology?” “What parallels might exist for this decorative motif?”

Risks: The system generates plausible archaeological or art-historical language without actual expertise. It may confidently misidentify objects or suggest parallels that don’t hold up. Useful for generating hypotheses to investigate, not for identification you can rely on.

Data Analysis and Visualization

Quantitative analysis

“I have enrollment data for 20 institutions across 10 years. Identify trends and create visualizations.” Systems with code execution can generate charts directly.

Risks: Statistical claims may be technically correct but misleading. Visualizations may emphasize patterns that aren’t significant or obscure patterns that are. If you’re not comfortable interpreting quantitative analysis, the system’s confident narrative may lead you astray. Know enough to interrogate what it produces, or consult someone who does.

Data transformation

Converting between formats, restructuring datasets, extracting structured information from unstructured text.

Risks: Subtle data corruption—fields shifted, values transformed incorrectly, edge cases handled badly. Spot-check outputs systematically, especially for large transformations where errors might hide in volume.

Prompting Principles

Specificity beats generality. Not “tell me about Virgil” but “identify three passages in Aeneid 6 where Virgil engages with Lucretian material, and explain what’s at stake in each engagement.”

Context constraints reshape outputs. “Without using psychoanalytic frameworks.” “Focusing on material evidence rather than literary sources.” “Assuming a reader skeptical of computational methods.”

Examples calibrate expectations. Show the structure, tone, or approach you want with one concrete instance.

Ask for alternatives. “Give me three different approaches” produces more useful material than asking for “the” answer.

Iterate with specific feedback. “This section works because X” “revise the rest to match.” “You’re treating the corpus as uniform, but periodization matters.”

Verification Strategies

Cross-model checking: Query multiple models (Claude, GPT, Gemini). Agreement increases confidence somewhat; disagreement definitely signals manual verification needed. But all models may share training-data errors, so agreement isn’t proof.

Citation verification: Assume all citations are potentially hallucinated until verified. Check that sources exist, that authors and titles match, that page numbers contain what’s claimed. This is non-negotiable for anything you’ll publish or submit.

Translation checking: For ancient languages, compare outputs across models and against published translations. Divergences flag passages requiring expert attention. Don’t trust any model’s translation as authoritative.

Advanced Use Cases

These require more setup or technical comfort but open significant possibilities for humanities work.

Document collections with NotebookLM

Google's [NotebookLM](#) offers the simplest entry point for working with your own document collections. Upload PDFs, Google Docs, or web pages (up to 50 sources), and the system lets you query across them, generate summaries, and explore connections "grounded in your materials rather than general training data.

Practical applications: uploading a reading list for a module and generating discussion questions grounded in the actual texts; loading project documentation and grant materials to query your own prior thinking; assembling sources for a literature review and asking for thematic analysis across them.

The system can also generate audio "podcasts" two AI voices discussing your materials in conversational format. This sounds gimmicky but has genuine accessibility applications, and some find audio summaries useful for initial orientation to unfamiliar material.

Risks: The system still generates text based on patterns, not understanding "it can misread sources, conflate distinct arguments, or produce confident synthesis that doesn't hold up on inspection. Grounding in your documents reduces hallucination but doesn't eliminate it. The audio feature can make unreliable content feel authoritative through the veneer of fluent conversation. Verify claims against your actual sources. Also note: your documents are processed on Google's servers.

Building lightweight RAG systems

RAG (Retrieval-Augmented Generation) combines search with generation: the system retrieves relevant passages from a document collection, then generates responses grounded in those passages. This allows working with corpora larger than any model's context window.

For humanities applications "personal archives, research collections, institutional documents" several tools offer accessible entry points without requiring programming:

[AnythingLLM](#) provides a desktop application for creating searchable document collections with various model backends. [Khoj](#) offers similar functionality with a focus on personal knowledge management. These tools handle document processing, chunking, and retrieval, letting you query your materials conversationally.

More technical users can build custom systems using embedding databases (Chroma, Pinecone) and frameworks like LangChain or LlamaIndex, allowing fine-grained control over how documents are processed and retrieved.

Risks: RAG systems can retrieve irrelevant passages, miss relevant ones, or generate responses that misrepresent retrieved content. Quality depends heavily on how documents are chunked and indexed "defaults may not suit humanities materials with complex structures. Setup requires technical investment, and maintenance isn't trivial. For most users, NotebookLM or similar managed services offer better effort-to-value ratios unless you have specific requirements they can't meet.

Media transformation

Current tools enable various transformations across media types:

Audio transcription: Services like [Whisper](#) (OpenAI, available free through various interfaces) transcribe audio with reasonable accuracy. Useful for interviews, oral histories, lecture recordings, archival audio. [Riverside](#), [Otter.ai](#), and others offer user-friendly interfaces.

Text to audio: NotebookLM's podcast feature, [ElevenLabs](#), and other text-to-speech services can convert written materials to audio. Applications include accessibility, creating audio versions of your own writing for review, or generating listening materials for students.

Video processing: Tools can transcribe video audio, generate summaries of video content, or extract key frames. YouTube's auto-generated transcripts, while imperfect, provide workable starting points for analysis of video materials.

Document conversion: Models can transform content between formats "converting narrative text to structured data, extracting tables from prose descriptions, reformatting archival documents into machine-readable formats.

Risks: Transcription errors compound through subsequent processing "a mistranscribed name becomes a hallucinated reference. Audio generation can make unreliable content feel polished and authoritative. Automated processing at scale can introduce systematic errors you won't catch without sampling. For anything you'll publish or cite, verify against originals.

Working with open-weights models

Frontier models from Anthropic, OpenAI, and Google are proprietary "you access them through APIs or chat interfaces, and your data passes through their servers. Open-weights models (Llama, Mistral, Qwen, and others) release their parameters publicly, allowing local installation or self-hosting.

Why consider this: complete privacy (nothing leaves your machine), no usage costs after setup, ability to fine-tune for specific tasks, and freedom from platform policy changes. [Hugging Face](#) hosts thousands of models with documentation and tools for experimentation.

For local use, [Ollama](#) provides the simplest setup—a single application that downloads and runs models locally with a chat interface. [LM Studio](#) offers a more visual interface. Both run on consumer hardware, though larger models require substantial RAM and benefit from GPU acceleration.

Risks: Open-weights models are generally less capable than frontier proprietary models—often significantly so. Local hardware constraints limit model size. Setup requires more technical comfort than using Claude or ChatGPT. Fine-tuning (training models on your own data) requires substantial technical expertise and computational resources. For most humanities users, the privacy benefits rarely outweigh the capability gap and setup costs—but for specific applications involving sensitive materials, or for experimentation and learning, local models offer valuable options.

Structured extraction at scale

Models can extract structured information from unstructured sources—named entities, dates, relationships, events—and output machine-readable formats (JSON, CSV, XML). Combined with batch processing, this enables working with document collections at scales impossible manually.

Applications: extracting person-place-date triples from historical correspondence; identifying and categorising marginalia across manuscript collections; building prosopographical databases from narrative sources; converting archival inventories into structured datasets.

This typically requires API access rather than chat interfaces, plus scripting to process documents in batches. Python libraries like [Instructor](#) simplify structured extraction. Costs scale with volume—budget accordingly.

Risks: Extraction errors multiply across large collections. Models may confidently extract “information” that isn’t there, impose anachronistic categories, or miss domain-specific entities. Quality control requires systematic sampling and validation against manual extraction. This is a power tool—powerful enough to produce large quantities of unreliable data if used carelessly. Start with small samples, validate thoroughly, and build in human review.

Custom assistants for projects

Most platforms allow creating specialised assistants with persistent instructions and uploaded reference materials. Claude offers “Projects” for this; ChatGPT has “Custom GPTs.”

You can create project-specific assistants preloaded with your research context, methodological preferences, key sources, and standing instructions. A “Roman epigraphy assistant” might include your transcription conventions, standard reference works, and instructions to flag dating uncertainties. A “thesis writing assistant” might include your chapter outlines, style preferences, and instructions to challenge weak arguments.

Risks: The assistant doesn’t actually understand your project—it generates contextually appropriate text based on your materials and instructions. Over-reliance on a well-configured assistant can create false confidence. The assistant may subtly reinforce your existing framing rather than challenging it, since it’s optimised to be helpful within your specified parameters. Use these as conveniences, not as substitutes for external feedback.

Voice capture and thought processing

Mobile apps for Claude, ChatGPT, and others accept voice input, enabling a workflow that separates idea generation from refinement: capture thoughts verbally, then use AI to process the raw material.

Two modes prove useful:

Quick capture: When an idea strikes—walking, commuting, between meetings—speak it into your phone. The app transcribes and you can later ask the system to extract key points, identify connections to ongoing work, or flag questions worth pursuing. This catches thinking that would otherwise evaporate.

Extended thinking aloud: Some find it productive to talk through a problem at length—fifteen minutes, half an hour—without worrying about structure or coherence. Rambling, backtracking, and contradiction are fine. Afterwards, ask the system to summarise the main threads, identify the core argument struggling to emerge, note tensions or unresolved questions, or restructure the content into an outline.

The voice-to-text-to-summary pipeline can feed into drafting, grant applications, lecture preparation, or simply clarifying your own thinking. Walking seems particularly conducive—the combination of movement, low-stakes verbal processing, and subsequent AI structuring works well for many people.

Risks: The summary is only as good as the raw material—if your rambling didn’t contain a coherent argument, the system will impose false coherence or surface something you didn’t actually mean. The system may emphasise what’s clearly articulated over what’s tentatively gestured at, when the tentative gesture might be the important part. Treat summaries as starting points for your own structuring, not as authoritative accounts of what you were thinking. Also consider privacy: if you’re speaking in public spaces, you’re potentially exposing research ideas; voice data passes through commercial servers like any other input.

When Not to Use These Tools

When verification would take longer than doing the task yourself. When you can’t verify the output’s accuracy. When the task requires genuine expertise you don’t have (the system’s confident output may mislead you). When the material is confidential and shouldn’t pass through commercial servers. When slow, careful engagement with materials is itself the point.

The question isn't whether these tools can produce output for a given task, but whether using them actually improves your work. Often the answer is no, and that's fine.

Glossary: AI Jargon and Acronyms

AGI (Artificial General Intelligence): Hypothetical AI that matches or exceeds human cognitive abilities across all domains. Does not currently exist; timelines for its development are contested. Often invoked in hype cycles. [MIT Technology Review explainer](#)

Alignment: The problem of ensuring AI systems do what their designers intend and behave in accordance with human values. An active research area, not a solved problem. [Anthropic's perspective](#)

API (Application Programming Interface): A way for software to communicate with other software. In AI contexts, this is how developers access models programmatically rather than through chat interfaces. Relevant if you're building tools; otherwise you'll use consumer interfaces.

Chatbot: Conversational interface to an AI system. ChatGPT, Claude, and Gemini are chatbots built on top of LLMs. The term predates modern AI—earlier chatbots used simpler rule-based systems.

Context window: The amount of text a model can process in a single conversation—both what you input and what it outputs. Measured in tokens. Current frontier models handle 100,000–200,000 tokens (roughly 75,000–150,000 words). Longer contexts allow working with entire documents but don't guarantee the model attends equally to all parts.

Embedding: A mathematical representation of text (or images) as coordinates in high-dimensional space, where similar meanings cluster together. Used for search and retrieval systems. [Vicki Boykis's accessible explanation](#)

Fine-tuning: Additional training of a foundation model on specific data to adapt it for particular tasks or domains. Creates specialized versions of general-purpose models.

Foundation model: A large model trained on broad data that serves as the base for many applications. GPT-4, Claude, and Gemini are foundation models. The term emphasizes that these are general-purpose starting points. [Stanford HAI overview](#)

Frontier model: Industry term for the most capable currently available models. What counts as "frontier" shifts as new models release.

Generative AI: AI systems that produce new content (text, images, audio, video) rather than classifying or analysing existing content. LLMs are generative AI for text.

GPT (Generative Pre-trained Transformer): OpenAI's series of language models (GPT-3, GPT-4, etc.). Often used loosely to mean any LLM, though technically it's a specific product line. The architecture (transformer) is shared across most current LLMs.

Guardrails: Restrictions built into AI systems to prevent harmful outputs. These include content filters, refusal behaviours, and output constraints. Effectiveness varies; determined users can often circumvent them.

Hallucination: When a model generates false information presented as fact—invented citations, incorrect dates, fictional events described confidently. A fundamental limitation of how these systems work, not a bug being fixed. [IBM explainer](#)

Inference: The process of a trained model generating outputs from inputs. When you prompt a model and it responds, that's inference. Distinct from training, which is how the model learned its patterns.

LLM (Large Language Model): AI systems trained on vast text corpora to predict and generate language. "Large" refers to the number of parameters (billions to trillions). Claude, GPT-4, and Gemini are LLMs. [Ars Technica accessible overview](#)

Multimodal: Models that process multiple types of input—text, images, audio, video—rather than text alone. Current frontier models are multimodal, accepting images and documents alongside text.

Neural network: Computing systems loosely inspired by biological brains, consisting of interconnected nodes that process information in layers. LLMs are a type of neural network. The "neural" metaphor is loose—these are mathematical functions, not simulated neurons.

Open source / Open weights: Models whose underlying parameters are publicly released, allowing others to run, modify, and build on them. Llama (Meta) and Mistral are prominent examples. "Open source" in AI is contested—releasing weights isn't the same as releasing training data or methods. [The Gradient on open source AI](#)

Parameters: The numerical values a model learns during training that determine its behaviour. Model size is often described by parameter count: GPT-4 reportedly has over a trillion parameters. More parameters generally means more capability but also more computational cost.

Prompt: The input you give to a model—your question, instruction, or the text you want it to work with. "Prompting" is the practice of crafting inputs to get useful outputs.

RAG (Retrieval-Augmented Generation): A technique combining LLMs with search systems. The model retrieves relevant documents from a database, then generates responses grounded in those documents. Reduces hallucination for domain-specific applications. [IBM explainer](#)

RLHF (Reinforcement Learning from Human Feedback): A training technique where human evaluators rate model outputs, and the model is trained to produce responses humans prefer. Used to make models more helpful and less harmful. [Anthropic explanation](#) | [Chip Huyen's accessible overview](#)

System prompt: Persistent instructions that shape how a model behaves across a conversation. Set by the platform or user to establish context, persona, or constraints. Distinct from individual prompts within a conversation.

Temperature: A setting controlling randomness in outputs. Lower temperature produces more predictable, focused responses; higher temperature produces more varied, creative ones. Default settings work for most uses.

Token: The unit models use to process text—roughly a word or word-piece. “Tokenization” breaks text into these units. Token counts matter for context windows and pricing. Roughly 1 token ≈ 0.75 words in English, though this varies.

Training data: The text (and images, for multimodal models) used to train a model. For LLMs, this includes large portions of the internet, books, code, and other text sources. What’s included in training data affects what models know and how they behave—and raises significant copyright and consent questions.

Transformer: The neural network architecture underlying most current LLMs, introduced in 2017. Enables models to process relationships between all parts of an input simultaneously rather than sequentially. [The Illustrated Transformer](#) (visual explanation, somewhat technical)

This document was developed collaboratively by Leif Isaksen and Claude (Anthropic).